# 单击查看速率预测数据处理和模型训练

## NetApp Solutions

NetApp
March 08, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/netapp-solutions/ai/aks-anf_libraries_for_data_processing_and_model_training.html on March 08, 2022. Always check docs.netapp.com for the latest.

# 目录

单击查看速率预测数据处理和模型训练 . . . . . 1
用于数据处理和模型培训的库 . . . . . 1
在熊猫中加载 Criteo 单击 Logs Day 15 ，然后训练一个 sc科学 学习随机林模型 . . . . . 1
在 dask 中加载第 15 天，训练一个 dask cuML 随机林模型 . . . . . 3
使用原生任务流信息板监控 dask . . . . . 5
培训时间比较 . . . . . 6
使用 Prometheus 和 Grafana 监控 dask 和快速流 . . . . . 7
使用 NetApp DataOps 工具包对数据集和模型进行版本控制 . . . . . 7
Jupyter 笔记本电脑作为参考 . . . . . 7

# 单击查看速率预测数据处理和模型训练

## 用于数据处理和模型培训的库

"先前版本： Azure NetApp Files 性能层。"

下表列出了用于构建此任务的库和框架。所有这些组件均已与 Azure 基于角色的访问和安全控制完全集成。

| 库 / 框架 | Description |
| --- | --- |
| dask cuML | 要使 ML 在 GPU 上运行，请使用 "cuML 库" 可通过 dask 访问快速通道 cML 软件包。与基于 CPU 的方法相比，强强联合可以通过基于 GPU 的高性能实施实施常见的 ML 算法，包括集群，维度缩减和回归方法，提供高达 100 倍的速度。 |
| dask cudf | CUDF 包括支持 GPU 加速提取，转换，加载（ ETL ）的各种其他功能，例如数据子设置，转换，单热编码等。快速发展团队会维护 "dask-cudf 库" 其中包括使用 dask 和 cuDF 的帮助程序方法。 |
| Scikit 学习 | Scikit Learning 提供了数十种内置机器学习算法和模型，称为评估器。每个 "估算器" 可以使用将其安装到某些数据中 方法 |

我们使用两台笔记本电脑构建 ML 管道进行比较；一台是传统的熊猫科学学习方法，另一台是使用快速和快速的分布式培训。每台笔记本电脑均可单独进行测试，以查看时间和规模方面的性能。我们会分别介绍每台笔记本电脑，以展示使用快速流和 dask 进行分布式培训的优势。

"接下来：在熊猫中加载 Criteo 单击 Logs Day 15 ，然后训练一个 sc科学 学习随机林模型。"

# 在熊猫中加载 Criteo 单击 Logs Day 15 ，然后训练一个 sc科学 学习随机林模型

"上一页：数据处理和模型培训库。"

本节介绍如何使用熊猫和 dask DataFrames 从 Criteo TB 数据集中加载 Click Logs 数据。在广告交换的数字广告中，使用情形与此相关，它可以预测是否会点击广告，或者如果交换在自动管道中使用的模型不准确，从而构建用户的个人资料。

我们从 Click Logs 数据集加载了第 15 天的数据，总计 45 GB 。在 Jupyter 笔记本电脑中运行以下单元 `CT-PandasRF-colled.ipynb` 创建一个包含前 5 ， 000 万行的熊猫 DataFrame ，并生成一个 scide-Learn 随机林模型。

```
%%time
import pandas as pd
import numpy as np
header = ['col'+str(i) for i in range (1,41)] #note that according to
criteo, the first column in the dataset is Click Through (CT). Consist of
40 columns
first_row_taken = 50_000_000 # use this in pd.read_csv() if your compute
resource is limited.
# total number of rows in day15 is 20B
# take 50M rows
"""
Read data & display the following metrics:
1. Total number of rows per day
2. df loading time in the cluster
3. Train a random forest model
"""
df = pd.read_csv(file, nrows=first_row_taken, delimiter='\t',
names=header)
# take numerical columns
df_sliced = df.iloc[:, 0:14]
# split data into training and Y
Y = df_sliced.pop('col1') # first column is binary (click or not)
# change df_sliced data types & fillna
df_sliced = df_sliced.astype(np.float32).fillna(0)
from sklearn.ensemble import RandomForestClassifier
# Random Forest building parameters
# n_streams = 8 # optimization
max_depth = 10
n_bins = 16
n_trees = 10
rf_model = RandomForestClassifier(max_depth=max_depth,
n_estimators=n_trees)
rf_model.fit(df_sliced, Y)
```

要使用经过培训的随机林模型执行预测，请在此笔记本电脑中运行以下段落。为了避免重复，我们采用了自第 15 天起的最后 100 万行作为测试集。该单元格还会计算预测准确性，其定义为模型准确预测用户是否单击 AD 。要查看此笔记本中任何不熟悉的组件，请参见 "官方科学知识工具包学习文档"。

```
# testing data, last 1M rows in day15
test_file = '/data/day_15_test'
with open(test_file) as g:
    print(g.readline())

# dataFrame processing for test data
test_df = pd.read_csv(test_file, delimiter='\t', names=header)
test_df_sliced = test_df.iloc[:, 0:14]
test_Y = test_df_sliced.pop('col1')
test_df_sliced = test_df_sliced.astype(np.float32).fillna(0)
# prediction & calculating error
pred_df = rf_model.predict(test_df_sliced)
from sklearn import metrics
# Model Accuracy
print("Accuracy:",metrics.accuracy_score(test_Y, pred_df))
```

"接下来：在 dask 中加载第 15 天，训练一个 dask cuML 随机林模型。"

# 在 dask 中加载第 15 天，训练一个 dask cuML 随机林模型

"Previous： Load Criteo click Logs Day 15 in 熊猫并训练一个 sc科学 学习随机林模型。"

按照与上一节类似的方式，在熊猫中加载 Criteo 单击 Logs Day 15 ，然后训练一个 cscit-Learn 随机林模型。在此示例中，我们使用 dask cuDF 执行了 DataFrame 加载，并在 dask cuML 中训练了一个随机林模型。我们比较了本节中培训时间和规模的差异 "" 培训时间比较。 ""

## Criteo_dask_rf.ipynb

此笔记本电脑将导入 `NumPy` ， **累积** 和必要的 `dask` 库，如以下示例所示：

```
import cuml
from dask.distributed import Client, progress, wait
import dask_cudf
import numpy as np
import cudf
from cuml.dask.ensemble import RandomForestClassifier as cumlDaskRF
from cuml.dask.common import utils as dask_utils
```

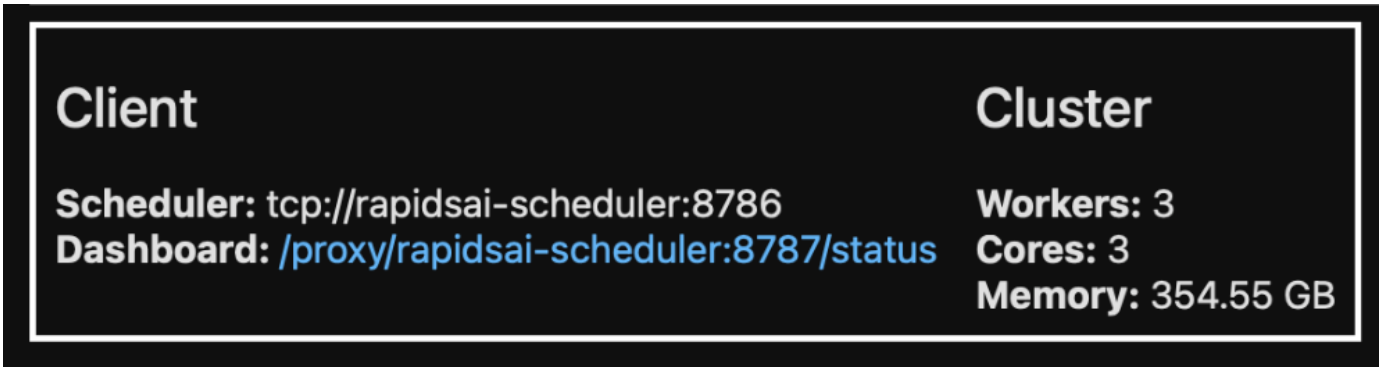
启动 dask 客户端（）。

```
client = Client()
```

如果集群配置正确，您可以查看工作节点的状态。

```
client
workers = client.has_what().keys()
n_workers = len(workers)
n_streams = 8 # Performance optimization
```

在我们的 AKS 集群中，将显示以下状态：




请注意， dask 采用了延迟执行模式： dask 不是即时执行处理代码，而是构建定向的环状图（ DAG ）执行。DAG 包含一组任务及其互动，每位员工都需要执行这些任务。此布局意味着，只有在用户指示 dask 以某种方式执行任务后，这些任务才会运行。使用 dask ，您有三个主要选项：

- 在 DataFrame 上调用 compute （）。 * 此调用将处理所有分区，然后将结果返回给计划程序，以便最终聚合和转换为 cuDF DataFrame 。除非计划程序节点内存不足，否则应谨慎使用此选项，并且仅在结果大幅减少时使用。
- * 在 DataFrame 上调用 persiste（ ）。 * 此调用执行图形，但它不会将结果返回到计划程序节点，而是在整个集群的内存中保留这些结果，以便用户可以在管道中重复使用这些中间结果，而无需重新运行相同的处理。
- * 在 DataFrame 上调用 head （）。 * 与 cuDF 一样，此调用会将 10 条记录返回到计划程序节点。此选项可用于快速检查 DataFrame 是否包含所需的输出格式，或者记录本身是否合理，具体取决于您的处理和计算结果。

因此，除非用户调用其中任一操作，否则员工将处于闲置状态，等待计划程序启动处理。这种延迟执行模式在现代并行和分布式计算框架（如 Apache Spark ）中很常见。

下一段使用 dask cuML 进行分布式 GPU 加速计算，以此训练随机林模型，并计算模型预测准确性。

```
Adsf
# Random Forest building parameters
n_streams = 8 # optimization
max_depth = 10
n_bins = 16
n_trees = 10
cuml_model = cumlDaskRF(max_depth=max_depth, n_estimators=n_trees,
n_bins=n_bins, n_streams=n_streams, verbose=True, client=client)
cuml_model.fit(gdf_sliced_small, Y)
# Model prediction
pred_df = cuml_model.predict(gdf_test)
# calculate accuracy
cu_score = cuml.metrics.accuracy_score( test_y, pred_df )
```

"下一步：使用原生任务流信息板监控 dask 。"

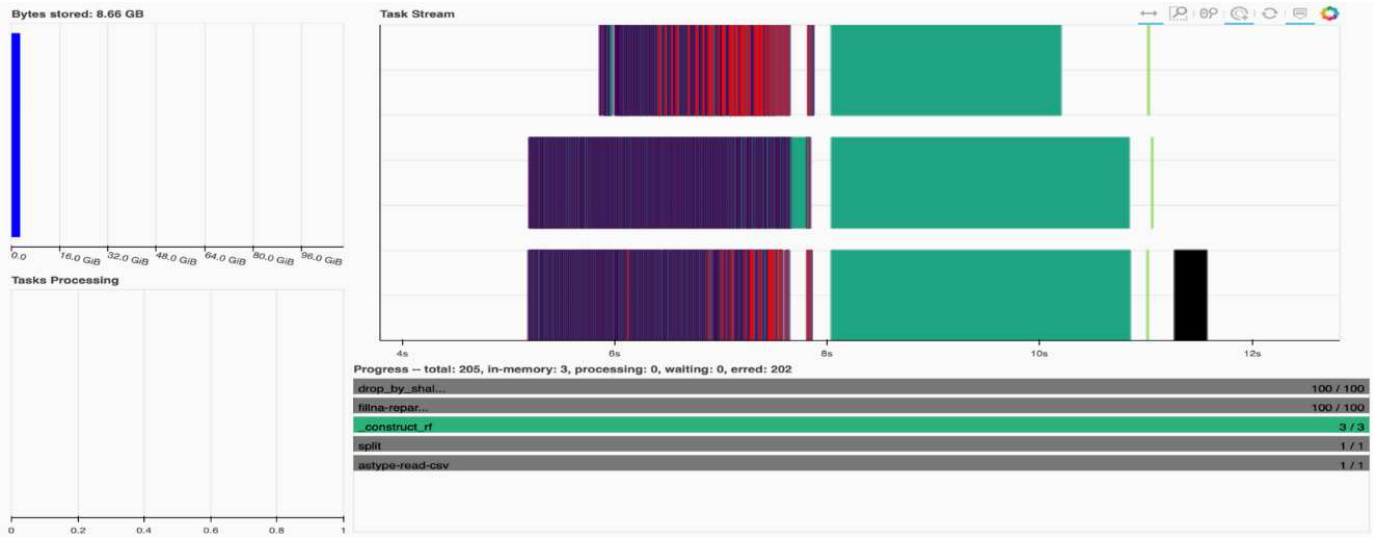
# 使用原生任务流信息板监控 dask

"上一步：在 dask 中加载 Day 15 并训练 dask cuML 随机林模型。"

。 "dask 分布式计划程序" 以两种形式提供实时反馈：

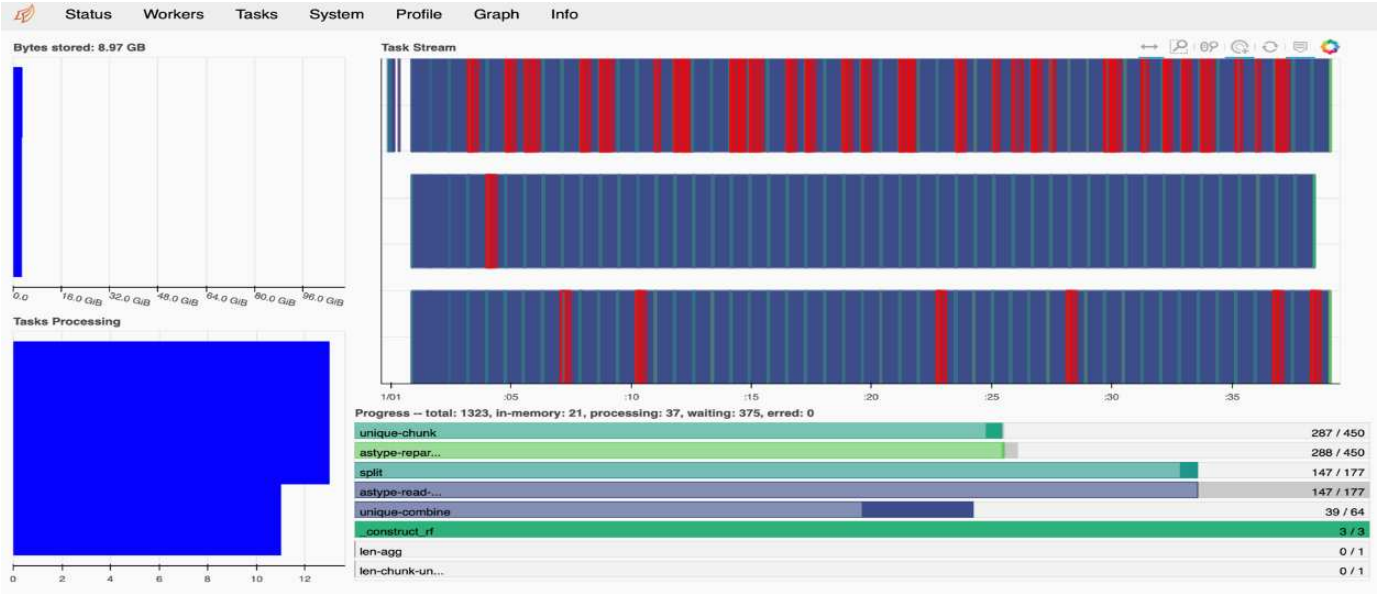
- 一个交互式信息板，其中包含许多图表和包含实时信息的表
- 一个适合在控制台或笔记本电脑中交互使用的进度条

在我们的案例中，下图显示了如何监控任务进度，包括存储的字节数，详细细分流数量的任务流以及执行关联功能的任务名称的进度。在我们的案例中，由于我们有三个辅助节点，因此流有三个主要区块，而颜色代码表示每个流中的不同任务。




您可以选择分析单个任务并以毫秒为单位检查执行时间，或者确定任何障碍或障碍。例如，下图显示了随机林模型安装阶段的任务流。要执行的功能要多得多，包括用于 DataFrame 处理的唯一区块，用于安装随机林的 _construct_rf 等。由于 Criteo Click Logs 中一天的数据非常大（ 45 GB ），因此大部分时间都花在了

DataFrame 操作上。




"下一步：比较训练时间。"

# 培训时间比较

"上一步：使用原生任务流信息板监控 dask 。"

本节将使用传统熊猫的模型训练时间与 dask 进行比较。对于熊猫，由于处理时间较慢，我们加载的数据较少，以避免内存溢出。因此，我们对结果进行了插值计算，以便进行合理的比较。

下表显示了当熊猫随机林模型（数据集每天 200 亿行中的 5000 万行）所使用的数据明显减少时的原始训练时间比较。 15此示例仅使用所有可用数据的 0.25% 以下。而对于 dask-cuML ，我们在所有 200 亿行可用的行上训练了随机林模型。这两种方法的训练时间相当。

| 方法 | 培训时间 |
| --- | --- |
| Scikit 学习：在 15 天仅使用 50 米行作为训练数据 | 47 分 21 秒 |
| rapids-dask ：使用第 15 天的所有 20 B 行作为训练数据 | 1 小时， 12 分钟和 11 秒 |

如果我们按线性方式插值训练时间结果，如下表所示，则使用分布式训练和 dask 具有显著优势。传统的熊猫科学学习方法需要 13 天来处理和训练 45 GB 的数据，只需一天的单击日志，而使用快速 dask 方法处理相同数量的数据则要快 262.39 倍。

| 方法 | 培训时间 |
| --- | --- |
| Scikit 学习：使用第 15 天的所有 20 B 行作为训练数据 | 13 天， 3 小时， 40 分钟和 11 秒 |
| rapids-dask ：使用第 15 天的所有 20 B 行作为训练数据 | 1 小时， 12 分钟和 11 秒 |

在上表中，您可以看到，通过使用带和 dask 的快速处理功能在多个 GPU 实例之间分布数据处理和模型训练，与使用 scide-Learn 模型训练的传统熊猫 DataFrame 处理相比，运行时间要短得多。此框架支持在云端以及多节点多 GPU 集群内部进行纵向和横向扩展。

"接下来：使用 Prometheus 和 Grafana 监控 dask 和快速流。"

# 使用 Prometheus 和 Grafana 监控 dask 和快速流

"上一步：训练时间比较。"

部署完所有内容后，对新数据运行推断。这些模型可根据浏览活动预测用户是否单击某个广告。预测结果存储在 dask cuDF 中。您可以使用 Prometheus 监控结果，并在 Grafana 信息板中直观显示结果。

有关详细信息，请参见此 "RAPID AI 中型 POST"。

"接下来：使用 NetApp DataOps 工具包进行数据集和型号版本控制。"

# 使用 NetApp DataOps 工具包对数据集和模型进行版本控制

"上一页：使用 Prometheus 和 Grafana 监控 dask 和 rapids 。"

适用于 Kubernetes 的 NetApp DataOps 工具包可将存储资源和 Kubernetes 工作负载抽象到数据科学工作空间级别。这些功能打包在一个简单易用的界面中，专为数据科学家和数据工程师设计。该工具包采用熟悉的 Python 计划形式，可帮助数据科学家和工程师在几秒钟内配置和销毁 JupyterLab 工作空间。这些工作空间可以包含数 TB 甚至数 PB 的存储容量，从而使数据科学家能够将其所有培训数据集直接存储在其项目工作空间中。单独管理工作空间和数据卷的时代已经一去不返。

有关详细信息，请访问工具包 "GitHub 存储库"。

"接下来：总结。"

# Jupyter 笔记本电脑作为参考

"先前版本：使用 NetApp DataOps 工具包的数据集和型号版本控制。"

本技术报告涉及两款 Jupyter 笔记本电脑：

- "* ct-andasrf-colled.ipynb.*" 此笔记本电脑可从 Criteo TB Click Logs 数据集加载第 15 天的数据，将数据处理并格式化到一个熊猫 DataFrame 中，训练一个 Scikit 学习随机林模型，执行预测并计算准确性。
- "* Criteo_dask_rf.ipynb.*" 此笔记本电脑可从 Criteo Terabyte Click Logs 数据集加载第 15 天数据，将数据处理并格式化为 dask cuDF ，训练 dask cuML 随机林模型，执行预测并计算准确性。这种分布式数据和模型处理和培训方法通过利用多个具有 GPU 的员工节点来实现高效率。与传统的 ML 方法相比，处理的数据越多，节省的时间就越多。您可以将此笔记本电脑部署在云，内部或混合环境中，其中 Kubernetes 集群包含不同位置的计算和存储，前提是您的网络设置可以自由移动数据和分发型号。

"接下来：总结。"

**Copyright Information**

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

**Trademark Information**

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.